Main page
- Getting started with STM32L4
  - Getting started with STM32 : STM32 step by step
  - Getting started with STM32 system peripherals
    - Getting started with GPIO
    - Getting started with EXTI
    - Getting started with PWR
    - Getting started with LPBAM
    - Getting started with DMA
    - Getting started with UART
    - Getting started with SPI
    - Getting started with I2C
    - Getting started with I3C
    - Getting started with TIM
    - Getting started with RTC
    - Getting started with WDG
    - Getting started with HRTIM
    - Getting started with DAC
    - Getting started with ADC
    - Getting started with Programmable Logic ArraY
- Getting started with STM32C5

Approved version. Approved on: 08:35, 17 June 2025

Pages Getting started with DMA

Last edited one year ago

Getting started with DMA

This article explains what DMA is and how to use it through examples.

Contents [hide]

1 What is DMA?
1.1 Objectives
1.2 DMA memory-to-memory example overview
2 Creating the project in STM32CubeIDE
3 Memory-to-memory mode
3.1 DMA process workflow summary
3.2 DMA M2M Configuration
3.3 Compile and flash
4 Memory-to-memory transfer with Interrupt
4.1 HAL Library DMA with IT flow
4.2 DMA M2M with IT Configuration
4.3 Compile and flash
5 References

1. What is DMA?↑

DMA stands for Direct Memory Access controller.
DMA is a bus master and system peripheral providing high-speed data transfers between peripherals and memory, as well as memory-to-memory.
Data can be quickly moved by DMA ***without any CPU action***, keeping CPU resources free for other operations.

This article uses the STM32L476 device as an example. The STM32L476 device embeds 2 DMAs: DMA1 and DMA2.

Each channel is dedicated to managing memory access requests from one or more peripherals. The two DMA controllers have 14 channels in total. Each channel has an arbiter to handle priority between DMA requests.

DMA features	DMA1	DMA2
Number of regular channels	7	7

1.1. Objectives↑

- Learning how to setup DMA transfer in STM32CubeIDE.
- Creating simple DMA memory-to-memory transfer from RAM to RAM and transfer with interrupt.

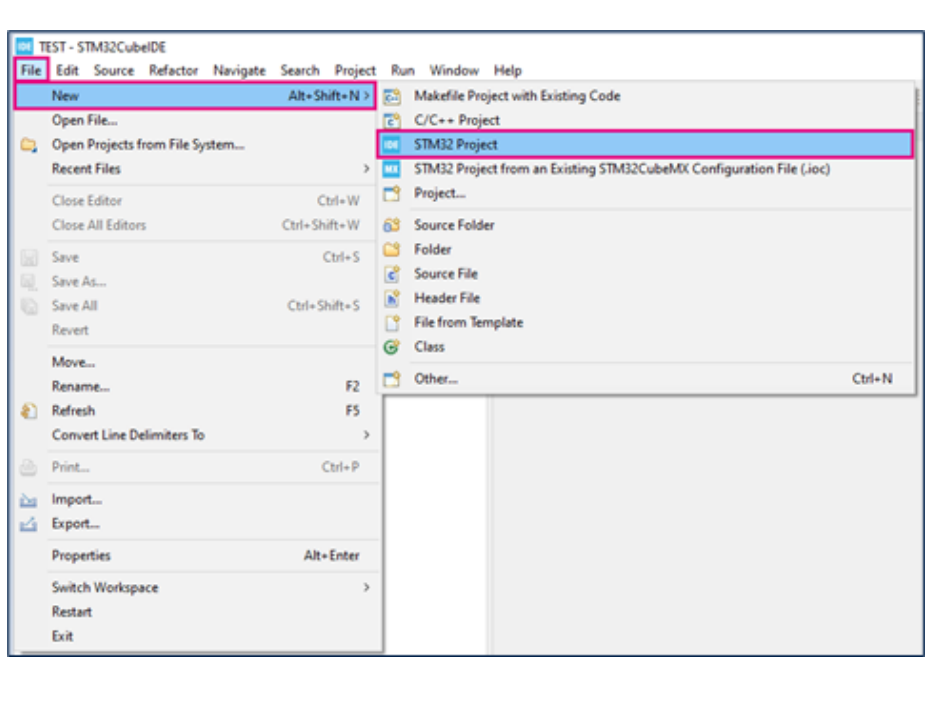
1.2. DMA memory-to-memory example overview↑

- Using STM32CubeIDE and generating code with DMA.
- Learning how to setup DMA using HAL.
- Verifying the correct functionality by comparing transferred buffers.

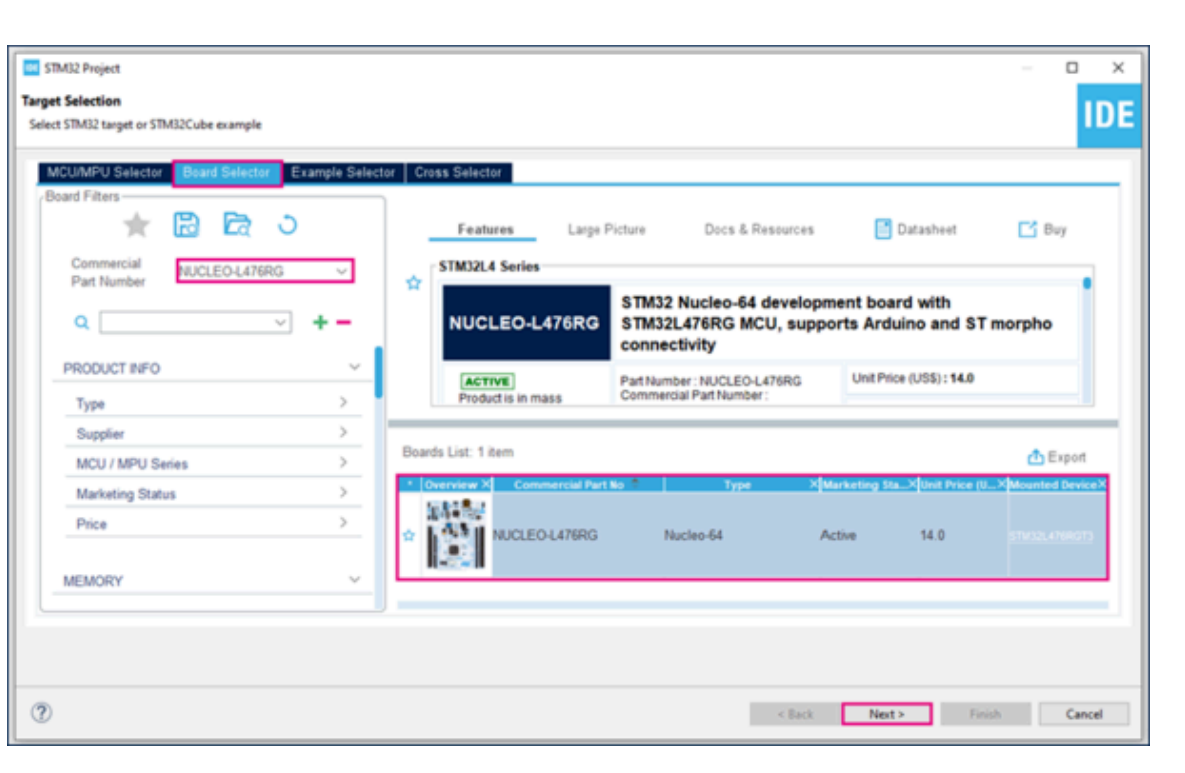
2. Creating the project in STM32CubeIDE↑

The example below uses the NUCLEO-L476RG board[1] .

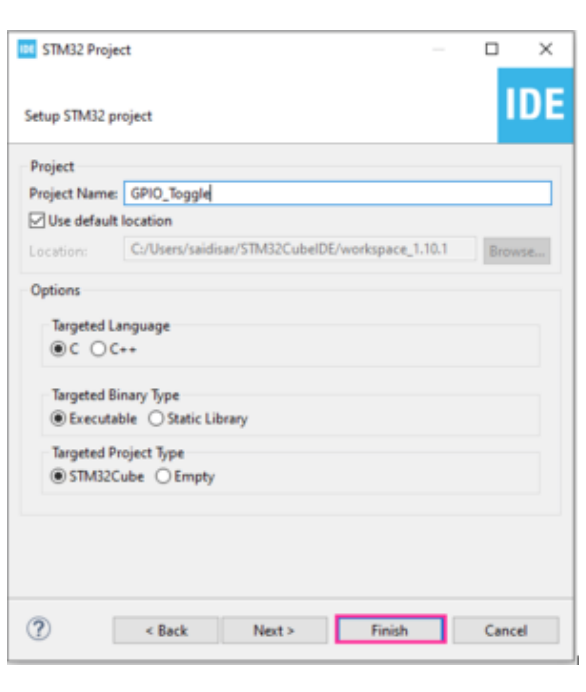
- ***File*** > ***New*** > ***STM32 Project*** in main panel.


File
New
Open File...
Makefile Project with Existing Code
C/C++ Project
STM32 Project
STM32 Project from an Existing STM32CubeMX Configuration File (.ioc)

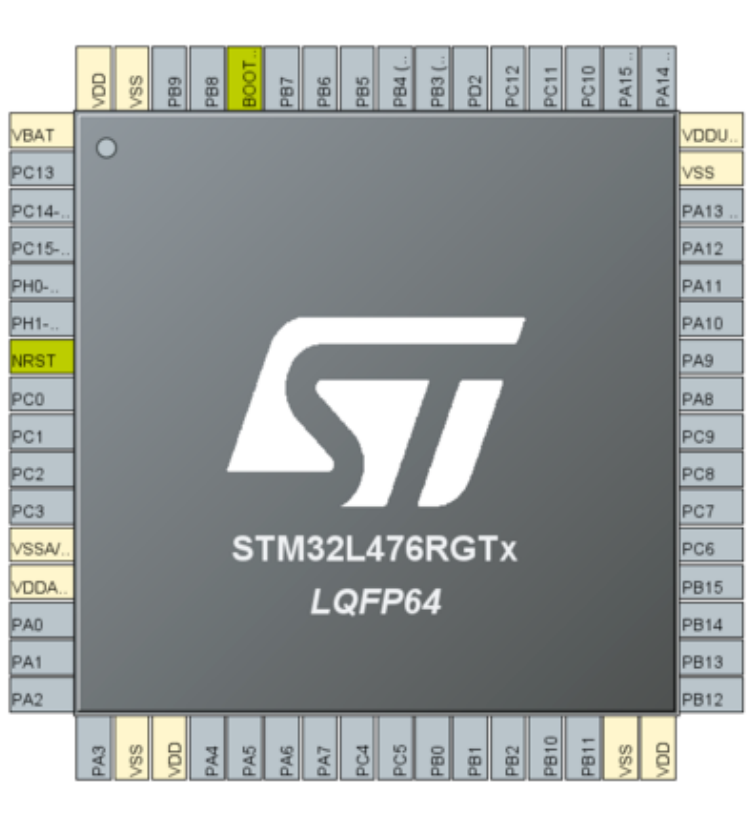
- Select the NUCLEO-L476RG board using the Board Selector, as shown in the figure below:


STM32 Project
Target Selection
IDE
MCU/MPU Selector
Board Selector
Example Selector
Cross Selector
NUCLEO-L476RG
STM32L4 Series
STM32 Nucleo-64 development board with STM32L476RG MCU, supports Arduino and ST morpho connectivity
Boards List: 1 item
NUCLEO-L476RG
Nucleo-64
Active
Next >
Cancel

- If not downloaded previously, the download of the STM32CubeL4 Cube library starts automatically. The download may take some time.
- Save the project.

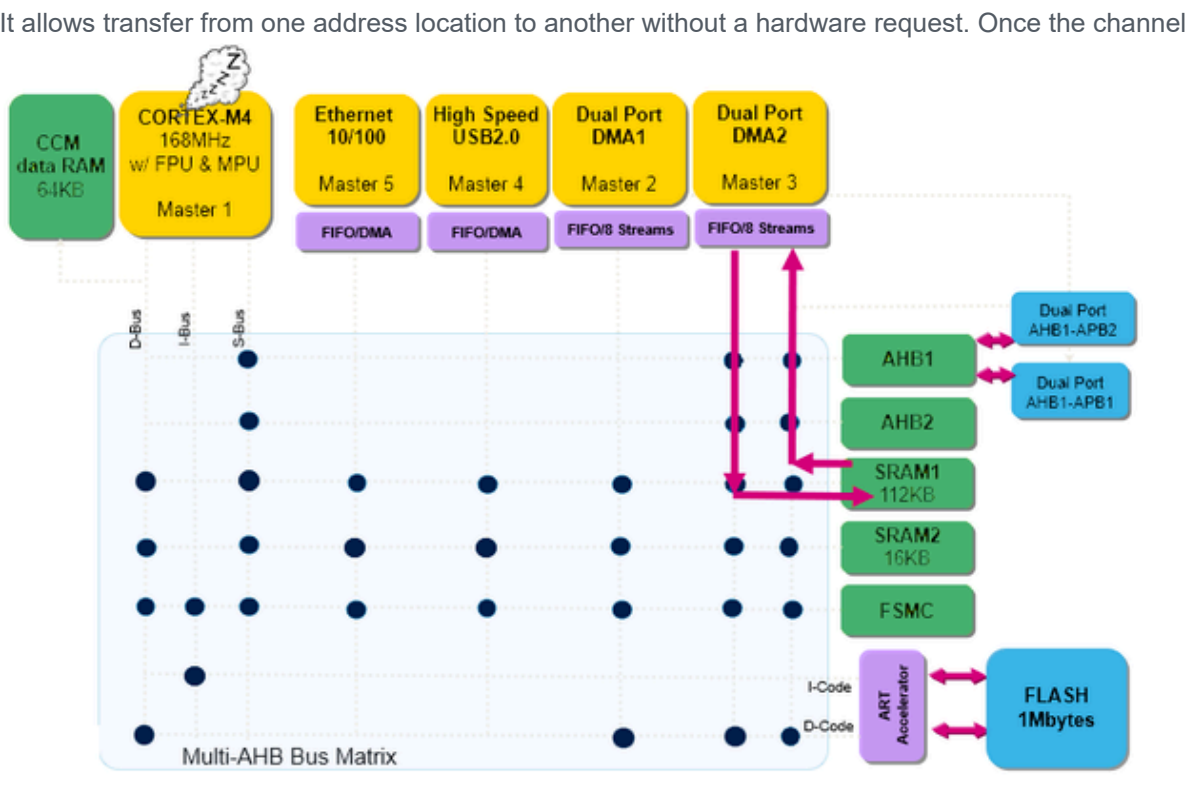

STM32 Project
Setup STM32 project
IDE
Project
Project Name: GPIO_Toggle
Use default location
Options
Targeted Language
C C++
Targeted Binary Type
Executable Static Library
Targeted Project Type
STM32Cube Empty
< Back
Next >
Finish
Cancel

- Pin configuration is not required for DMA.


STM32L476RGTx
LQFP64

3. Memory-to-memory mode↑

DMA channels may operate without being triggered by a request from a peripheral. This mode is called memory-to-memory mode, and is initiated by software.
It allows transfer from one address location to another without a hardware request. Once the channel is configured and enabled, the transfer starts immediately.

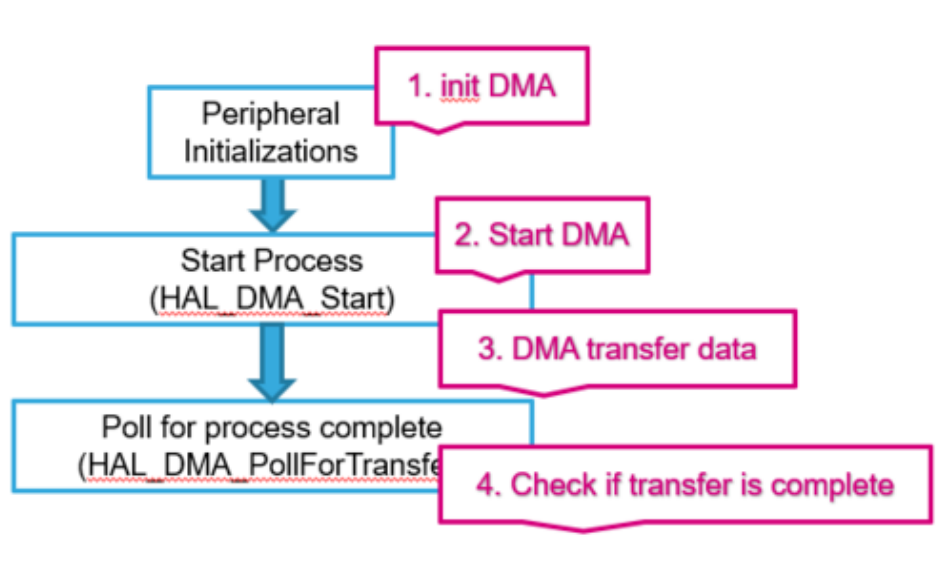

CCM data RAM 64KB
CORTEX-M4 168MHz w/ FPU & MPU
Master 1
Ethernet 10/100
Master 5
High Speed USB2.0
Master 4
Dual Port DMA1
Master 2
Dual Port DMA2
Master 3
FIFO/DMA
FIFO/DMA
FIFOS Streams
FIFOS Streams
D-Bus
I-Bus
S-Bus
AHB1
AHB2
SRAM1 112KB
SRAM2 16KB
FSMC
Dual Port AHB1-APB2
Dual Port AHB1-APB1
I-Code
D-Code
ART Accelerator
FLASH 1Mbytes
Multi-AHB Bus Matrix

Warning

Memory-to-memory mode must not be used in circular mode.

3.1. DMA process workflow summary↑

- At the beginning of the main program, HAL_DMA_Init() is called to reset all peripherals, initialize flash interface and systick.
- HAL_DMA_Start() starts the DMA transfer after the configuration of source and destination addresses, as well as the length of data to be transferred.
- HAL_DMA_PollForTransfer() polls for the end of current transfer. In this case a fixed timeout can be configured by user depending on his application.

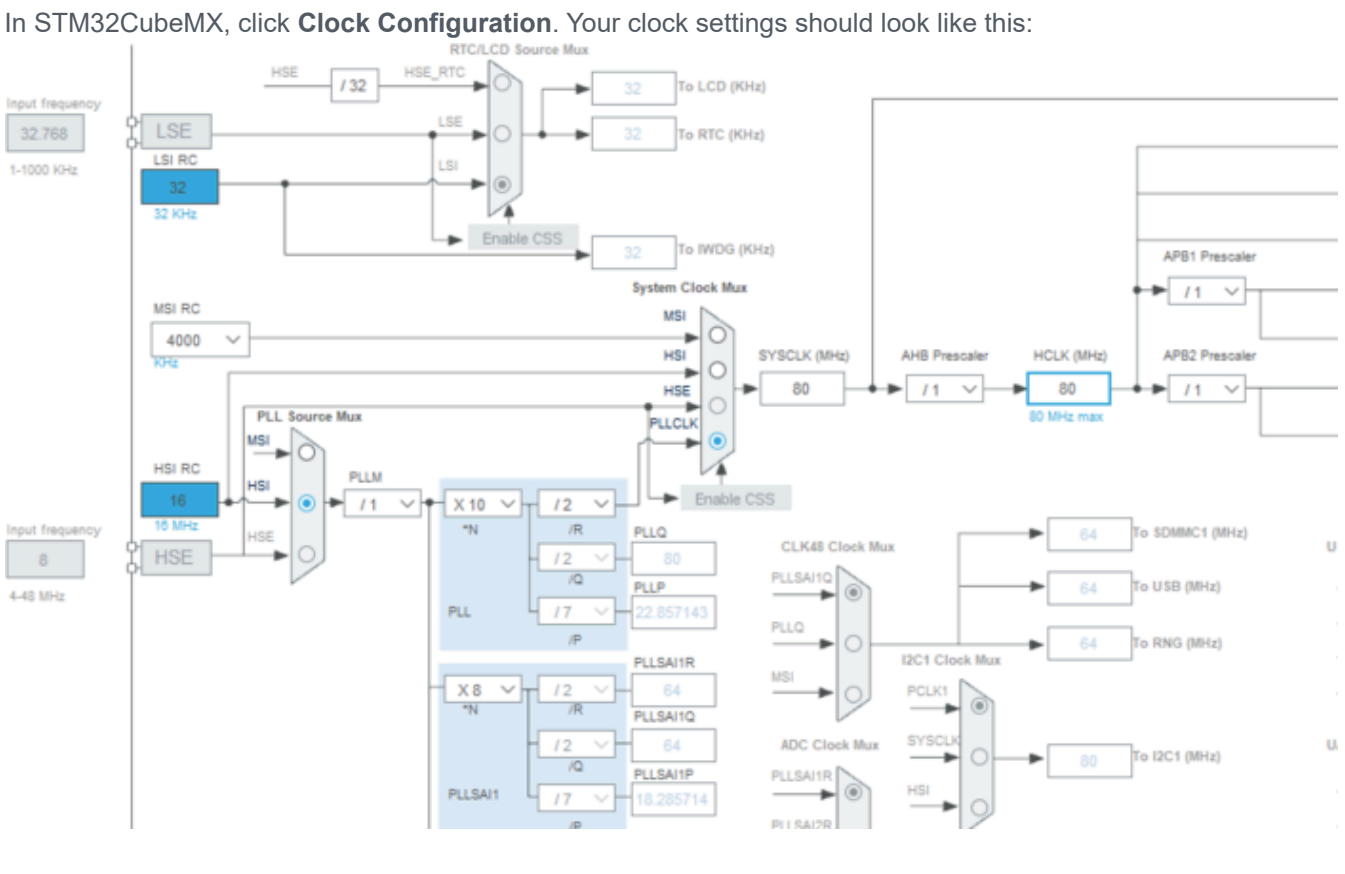

Peripheral Initializations
1. init DMA
Start Process (HAL_DMA_Start)
2. Start DMA
3. DMA transfer data
Poll for process complete (HAL_DMA_PollForTransfer)
4. Check if transfer is complete

Once the transfer is completed, it is recommended to handle the return values to make sure the program worked as expected.

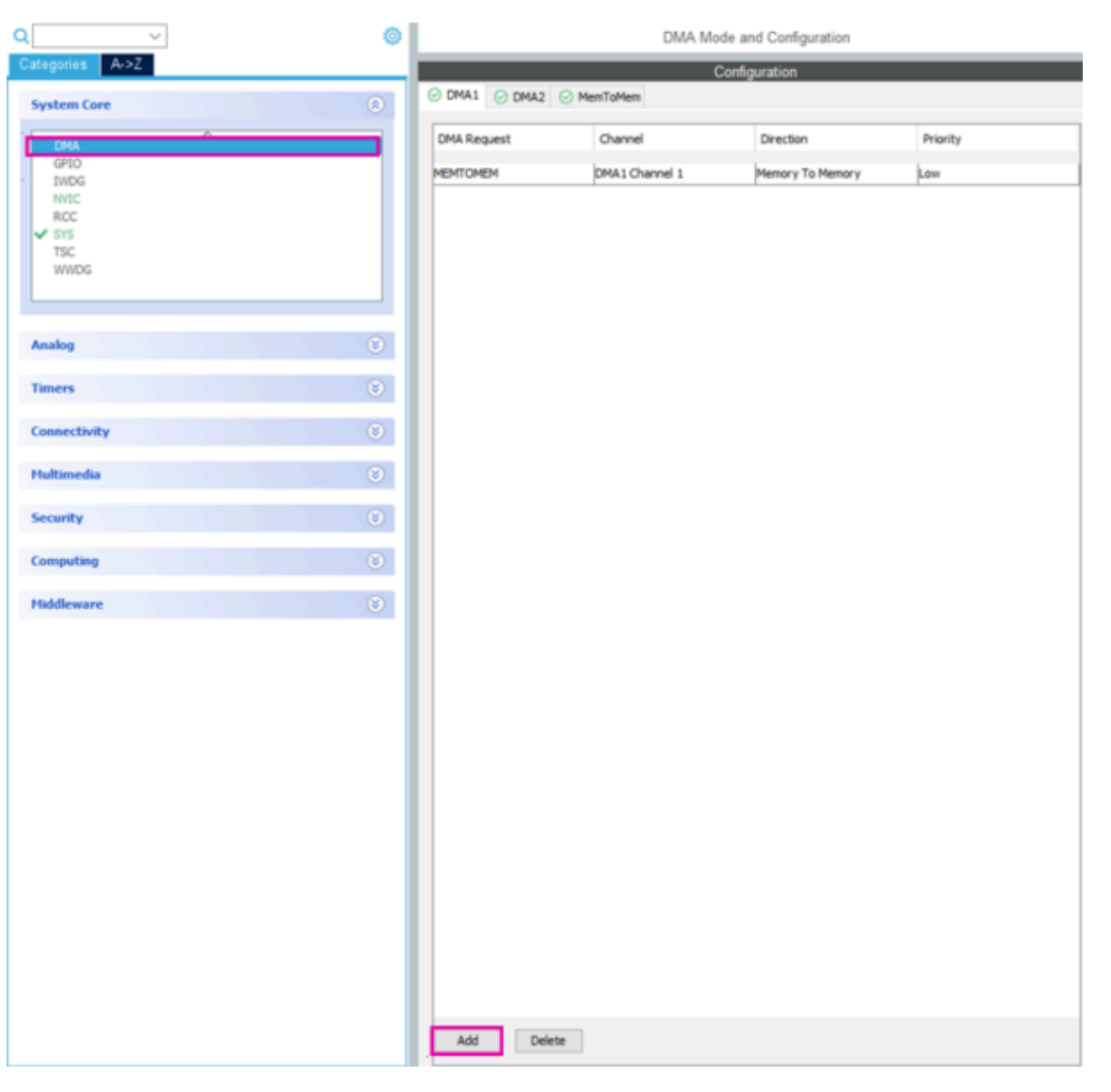
- HAL_OK: DMA transfer was successfully finished and data was transferred to destination without error.
- HAL_ERROR: Error occured during DMA transfer. You can use HAL_DMA_GetError for details.
- HAL_BUSY: DMA transfer in progress, user can only abort the transfer

3.2. DMA M2M Configuration↑

In STM32CubeMX, click **Clock Configuration**. Your clock settings should look like this:

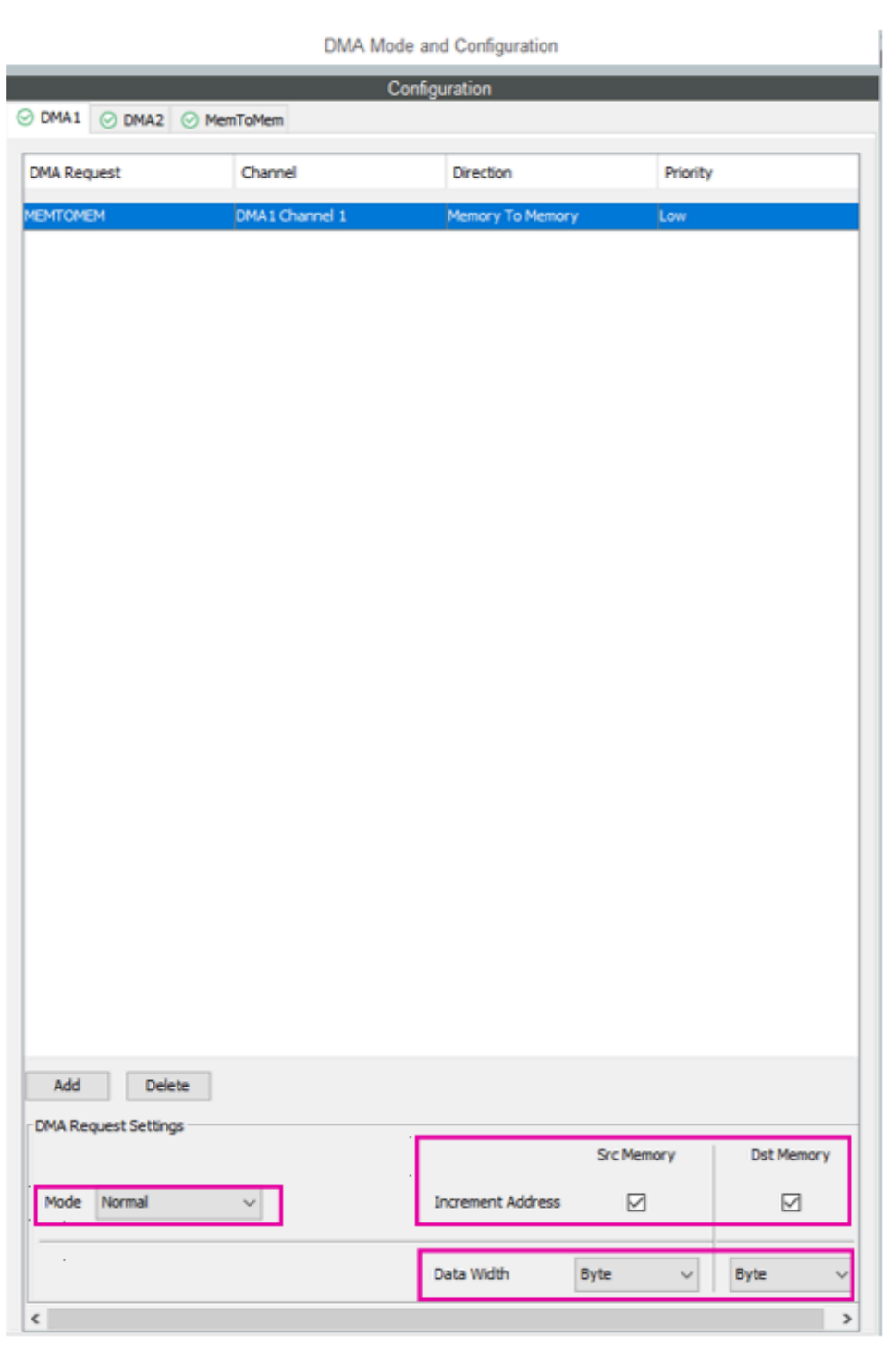

Input frequency
32.768
LSE
LSI RC
32
HSI RC
16
MSI RC
4000
HSE
PLL Source Mux
System Clock Mux
SYSCLK (MHz)
80
AHB Prescaler
HCLK (MHz)
80
APB1 Prescaler
APB2 Prescaler
PLLM
PLLN
PLLR
PLLQ
PLLP
PLLSAI1
PLLSAI2
To SDMMC1 (MHz)
To USB (MHz)
To RNG (MHz)
To SAI1 (MHz)

Now, click **Pinout & Configuration** > **DMA** > **DMA1** Button **Add**.

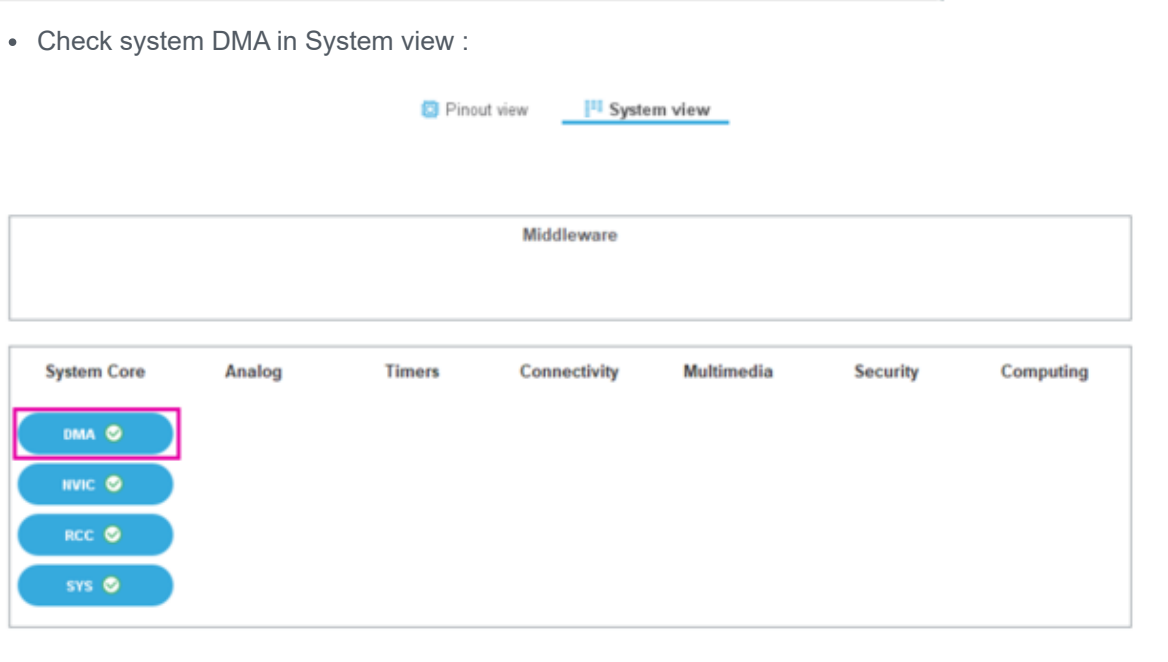

Categories
A->Z
System Core
DMA
GPIO
IWDG
NVIC
RCC
SYS
TSC
WWDG
Analog
Timers
Connectivity
Multimedia
Security
Computing
Middleware
DMA Mode and Configuration
Configuration
DMA1
DMA2
MemToMem
DMA Request
Channel
Direction
Priority
MEMTOMEM
DMA1 Channel 1
Memory To Memory
Low
Add
Delete

Configure DMA as follows:

- MEMTOMEM DMA request : DMA1 Channel 1
- Normal mode
- Increment source and destination addresses
- Byte data width


DMA Mode and Configuration
Configuration
DMA1
DMA2
MemToMem
DMA Request
Channel
Direction
Priority
MEMTOMEM
DMA1 Channel 1
Memory To Memory
Low
Add
Delete
DMA Request Settings
Mode
Normal
Src Memory
Dst Memory
Increment Address
☑
☑
Data Width
Byte
Byte

- Check system DMA in System view :


Pinout view
System view
Middlewares
System Core
Analog
Timers
Connectivity
Multimedia
Security
Computing
DMA
NVIC
RCC
SYS

- Generate the code by pressing ***Ctrl + S***.
- Open ***main.c*** using the Project Explorer: ***myproject / Src / main.c***.
- Create two buffers: the first for source data, and the second as destination buffer.

Information

Insert your code between the tags **/* USER CODE BEGIN 0 */** and **/* USER CODE END 0 */**

```
/* USER CODE BEGIN 0 */
uint8_t Buffer_Src[]={0,1,2,3,4,5,6,7,8,9};
uint8_t Buffer_Dest[10];
/* USER CODE END 0 */
```

Information

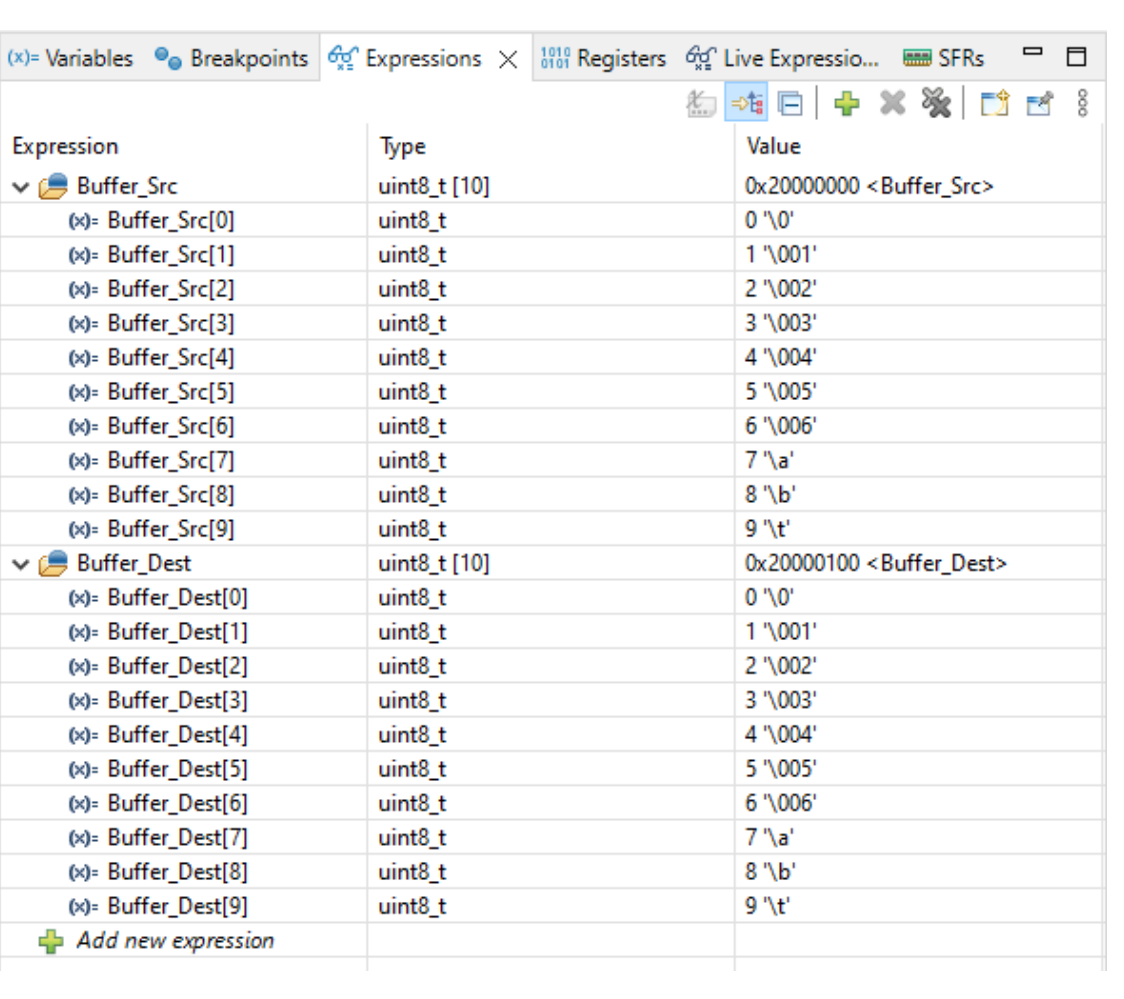
Insert your code between the tags **/* USER CODE BEGIN 2 */** and **/* USER CODE END 2 */**

```
/* USER CODE BEGIN 2 */
HAL_DMA_Start(&hdma_memtomem_dma1_channel1, (uint32_t) (Buffer_Src), (uint32_t) (Buffer_Dest), 10);
while(HAL_DMA_PollForTransfer(&hdma_memtomem_dma1_channel1, HAL_DMA_FULL_TRANSFER, 100) != HAL_OK)
{
    __NOP();
}
/* USER CODE END 2 */
```

3.3. Compile and flash↑

- Click ***Debug*** button to run step by step
- Click ***Resume*** to continue execution
- Click ***Suspend*** to stop execution

Add *Buffer_Src* and *Buffer_Dest* to *Expressions*, on the right side of STM32CubeIDE, to monitor their value. Source data has been transferred to destination buffer.

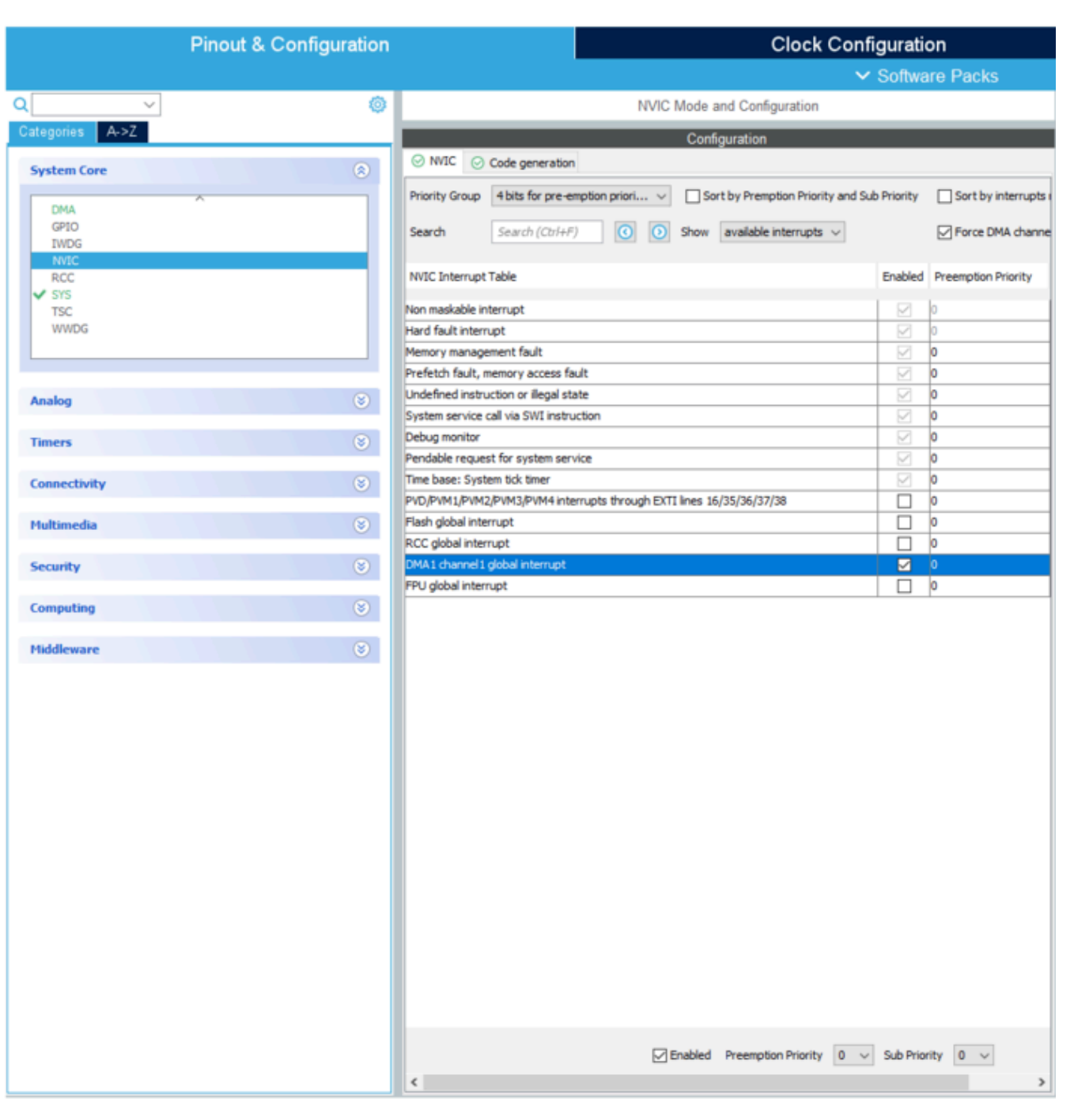

Variables
Breakpoints
Expressions
Registers
Live Expressions
SFRs
Add new expression

Expression	Type	Value
Buffer_Src	uint8_t [10]	0x20000000 <Buffer_Src>
Buffer_Src[0]	uint8_t	0 '\0'
Buffer_Src[1]	uint8_t	1 '\001'
Buffer_Src[2]	uint8_t	2 '\002'
Buffer_Src[3]	uint8_t	3 '\003'
Buffer_Src[4]	uint8_t	4 '\004'
Buffer_Src[5]	uint8_t	5 '\005'
Buffer_Src[6]	uint8_t	6 '\006'
Buffer_Src[7]	uint8_t	7 '\a'
Buffer_Src[8]	uint8_t	8 '\b'
Buffer_Src[9]	uint8_t	9 '\t'
Buffer_Dest	uint8_t [10]	0x20000100 <Buffer_Dest>
Buffer_Dest[0]	uint8_t	0 '\0'
Buffer_Dest[1]	uint8_t	1 '\001'
Buffer_Dest[2]	uint8_t	2 '\002'
Buffer_Dest[3]	uint8_t	3 '\003'
Buffer_Dest[4]	uint8_t	4 '\004'
Buffer_Dest[5]	uint8_t	5 '\005'
Buffer_Dest[6]	uint8_t	6 '\006'
Buffer_Dest[7]	uint8_t	7 '\a'
Buffer_Dest[8]	uint8_t	8 '\b'
Buffer_Dest[9]	uint8_t	9 '\t'

This method requires polling for transfer status. The method to achieve memory-to-memory transfer with interrupts is described below.

4. Memory-to-memory transfer with Interrupt↑

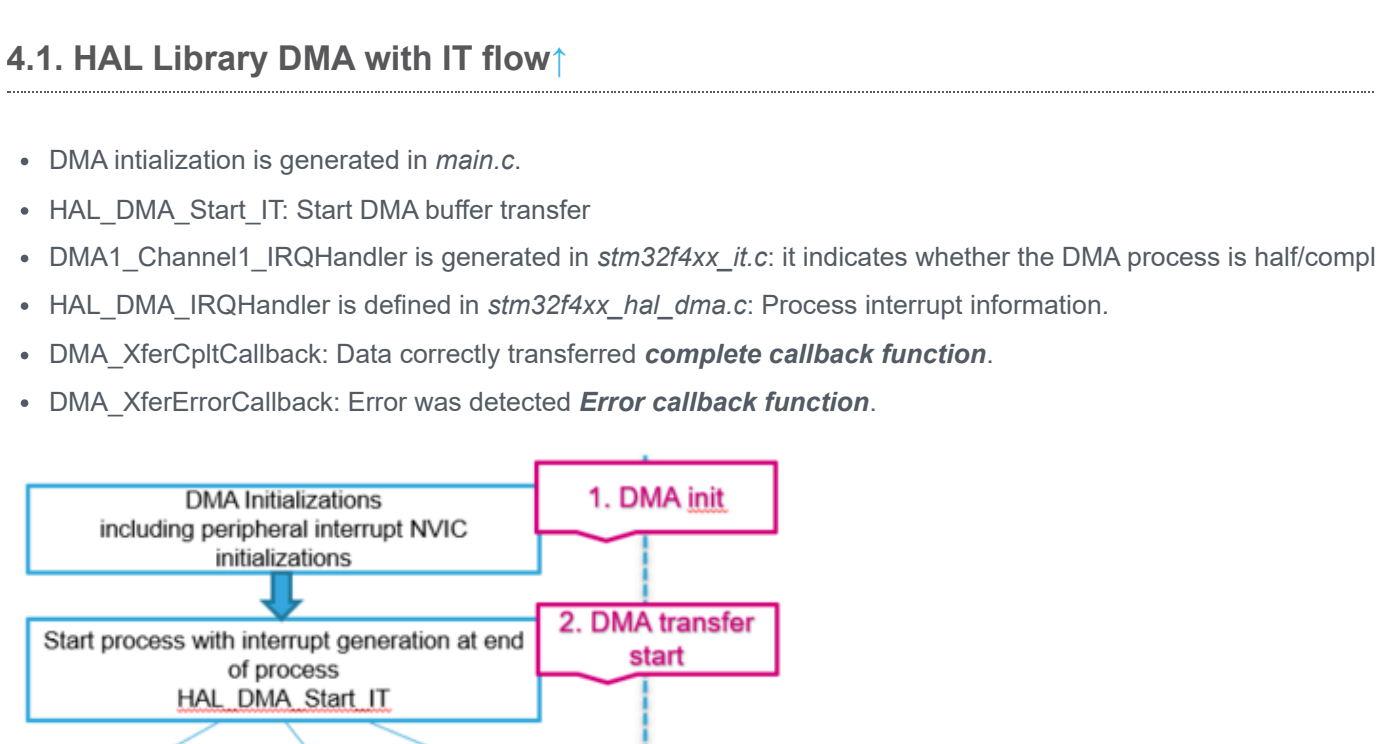
- To configure DMA with Interrupt, follow the steps detailed in the Memory-to-memory mode section.
- Enable **DMA1 Channel 1 Global Interrupt** in **System Core > NVIC** as shown in the figure below:


Pinout & Configuration
Clock Configuration
Software Packs
Categories
A->Z
System Core
DMA
GPIO
IWDG
NVIC
RCC
SYS
TSC
WWDG
Analog
Timers
Connectivity
Multimedia
Security
Computing
Middleware
NVIC Mode and Configuration
Configuration
NVIC
Code generation
Priority Group
4 bits for pre-emption prior...
Sort by Premption Priority and Sub Priority
Sort by interrupts
Search
Show
available interrupts
Force DMA channels
NVIC Interrupt Table
Enabled
Preemption Priority
Non maskable interrupt
Hard fault interrupt
Memory management fault
Prefetch fault, memory access fault
Undefined instruction or illegal state
System service call via SWI instruction
Debug monitor
Pendable request for system service
Time base: System tick timer
PVD/PVM1/PVM2/PVM3/PVM4 interrupts through EXTI lines 16/35/36/37/38
Flash global interrupt
RCC global interrupt
DMA1 channel1 global interrupt
FPU global interrupt
Enabled
Preemption Priority
Sub Priority

- Now generate the code by pressing : ***Ctrl + S***

4.1. HAL Library DMA with IT flow↑

- DMA initialization is generated in *main.c*.
- HAL_DMA_Start_IT: Start DMA buffer transfer
- DMA1_Channel1_IRQHandler is generated in *stm32f4xx_it.c*: it indicates whether the DMA process is half/complete or an error was detected.
- HAL_DMA_IRQHandler is defined in *stm32f4xx_hal_dma.c*: Process interrupt information.
- DMA_XferCpltCallback: Data correctly transferred ***complete callback function***.
- DMA_XferErrorCallback: Error was detected ***Error callback function***.


DMA Initializations including peripheral interrupt NVIC initializations
1. DMA init
Start process with interrupt generation at end of process HAL_DMA_Start_IT
2. DMA transfer start
HAL_OK
HAL_ERROR
HAL_BUSY
HAL_DMA_IRQHandler
DMA1_Channel1_IRQHandler
3. DMA transfer complete or error
4. HAL DMA management
end of process callback DMA_XferCpltCallback
5. DMA transfer was correct
process Error callback DMA_XferErrorCallback
5. Error in DMA transfer

4.2. DMA M2M with IT Configuration↑

We will be using the same code as in DMA M2M Configuration.

Information

Insert your code between the tags **/* USER CODE BEGIN 0 */** and **/* USER CODE END 0 */**

```
/* USER CODE BEGIN 0 */
void XferCpltCallback(DMA_HandleTypeDef *hdma);
uint8_t Buffer_Src[]={0,1,2,3,4,5,6,7,8,9};
uint8_t Buffer_Dest[10];
/* USER CODE BEGIN 0 */
```

Information

Insert your code between the tags **/* USER CODE BEGIN 4 */** and **/* USER CODE END 4 */**

```
/* USER CODE BEGIN 4 */
void XferCpltCallback(DMA_HandleTypeDef *hdma)
{
  __NOP(); //Line reached only if transfer was successful. Toggle a breakpoint here
}
/* USER CODE END 4 */
```

Before we start the DMA with interrupt, we need to set the callback into DMA structure.
Then, it is possible to use the HAL_DMA_Start_IT to begin DMA transfer.

Information

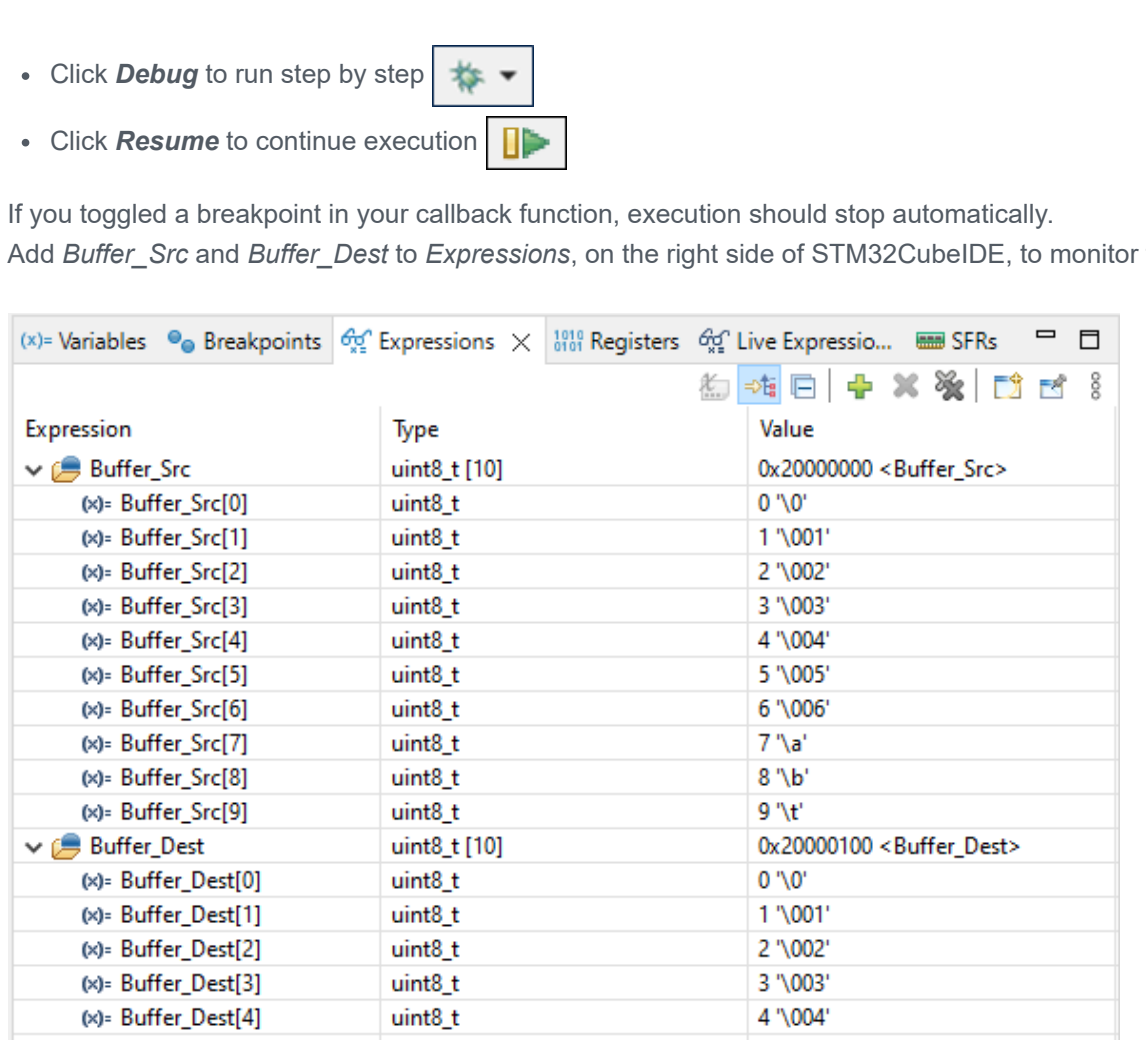
Insert your code between the tags **/* USER CODE BEGIN 2 */** and **/* USER CODE END 2 */**

```
/* USER CODE BEGIN 2 */
hdma_memtomem_dma1_channel1.XferCpltCallback=&XferCpltCallback;
HAL_DMA_Start_IT(&hdma_memtomem_dma1_channel1,(uint32_t)Buffer_Src,(uint32_t)Buffer_Dest,10);
/* USER CODE END 2 */
```

4.3. Compile and flash↑

- Click ***Debug*** to run step by step
- Click ***Resume*** to continue execution

If you toggled a breakpoint in your callback function, execution should stop automatically.
Add *Buffer_Src* and *Buffer_Dest* to *Expressions*, on the right side of STM32CubeIDE, to monitor their value. Source data has been transferred to destination buffer.


Variables
Breakpoints
Expressions
Registers
Live Expressions
SFRs
Add new expression

Expression	Type	Value
Buffer_Src	uint8_t [10]	0x20000000 <Buffer_Src>
Buffer_Src[0]	uint8_t	0 '\0'
Buffer_Src[1]	uint8_t	1 '\001'
Buffer_Src[2]	uint8_t	2 '\002'
Buffer_Src[3]	uint8_t	3 '\003'
Buffer_Src[4]	uint8_t	4 '\004'
Buffer_Src[5]	uint8_t	5 '\005'
Buffer_Src[6]	uint8_t	6 '\006'
Buffer_Src[7]	uint8_t	7 '\a'
Buffer_Src[8]	uint8_t	8 '\b'
Buffer_Src[9]	uint8_t	9 '\t'
Buffer_Dest	uint8_t [10]	0x20000100 <Buffer_Dest>
Buffer_Dest[0]	uint8_t	0 '\0'
Buffer_Dest[1]	uint8_t	1 '\001'
Buffer_Dest[2]	uint8_t	2 '\002'
Buffer_Dest[3]	uint8_t	3 '\003'
Buffer_Dest[4]	uint8_t	4 '\004'
Buffer_Dest[5]	uint8_t	5 '\005'
Buffer_Dest[6]	uint8_t	6 '\006'
Buffer_Dest[7]	uint8_t	7 '\a'
Buffer_Dest[8]	uint8_t	8 '\b'
Buffer_Dest[9]	uint8_t	9 '\t'

5. References↑

1. ↑ NUCLEO-L476RG board (64-pin)
- AN4031

Getting started with STM32 system peripherals

All rights reserved (c) 2026 STMicroelectronics